

1A) I created a database called northwind in MongoDB Atlas and loaded each CSV file (categories, customers, employees, orders, products, suppliers) into its own collection using a small helper function in Python that reads the file into a pandas DataFrame and then converts each row to a dictionary. Before inserting, I clear the collections with `delete_many({})` so that I can re-run the script without duplicating documents. For the “base” entities (categories, customers, employees, suppliers), I insert them first and build simple Python dictionaries that map the original relational IDs (like CategoryID, CustomerID, etc.) to the new MongoDB `_id` values. In the case of categories, I also keep a copy of the category row (without CategoryID) so I can embed that later.

When inserting products, I model the relationship with categories and suppliers in two different ways. For each product, I look up its CategoryID in my `category_doc_map` and embed the whole category document into a `category` field inside the product document. Categories are small, descriptive, and quite stable, and they’re almost always accessed through products, so embedding them makes reads simpler and avoids extra queries or joins. On the other hand, I keep suppliers as a separate collection and store only a `SupplierId` field in the product, which is an ObjectId reference to the supplier collection. Suppliers can be shared across many products and might need to be updated independently, so referencing them instead of embedding avoids a lot of duplication and makes updates easier.

For orders, I keep them as separate documents in the orders collection and treat them as the “linking” entity between customers, employees and products. When I insert each order, I replace the original foreign key fields (CustomerID, EmployeeID, ProductID) with the corresponding ObjectIds from `customer_id_map`, `employee_id_map` and `product_id_map`, storing them as `CustomerId`, `EmployeeId` and `ProductId`. In other words, orders reference their related customer, employee and product instead of embedding them. This avoids duplicating full customer/employee/product data in every single order and matches the idea that those entities are central and can appear in many orders. So overall, I embed small, mostly-read-only data (category inside product) and use ObjectId references for entities that are shared and updated across many documents (suppliers, customers, employees, products).

1B) This query lists, for each supplier, all products they supply together with unit prices. I start from the supplier collection and use a `$lookup` into products, matching the supplier’s `_id` with the `SupplierId` field stored in each product document. This recreates the relational join between suppliers and products. The resulting matched products are stored in a products array. In the `$project` stage, I hide the internal `_id` and use `$map` over this array to keep only the product name and unit price. The final output has one document per supplier with a `SupplierName` and a tidy `Products` list of (`ProductName`, `UnitPrice`) pairs.

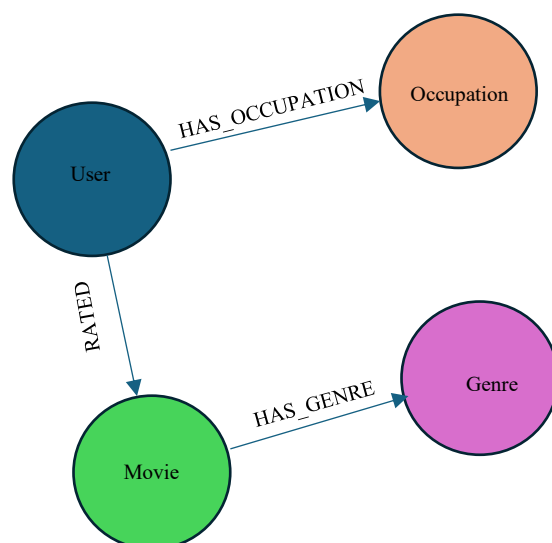
1C) This query builds, for each customer, a list of products they have purchased and how much they spent on each. I first join orders with the customers collection using `$lookup` on `CustomerId` and `_id`, and then `$unwind` the customer array so each order has a single embedded customer document. I repeat the pattern with `$lookup` and `$unwind` to join each order to the corresponding product in the products collection via `ProductId`. Next, I group by the pair (`customerId`, `productId`) to compute `totalQuantity` and `totalSpent` per product per customer, using $\text{Quantity} \times \text{UnitPrice} \times (1 - \text{Discount})$ to incorporate discounts. A second `$group` then groups by customer only, pushing the per-product information into a products array and summing `totalSpent` to obtain `CustomerTotalSpent`. The final `$project` removes the technical `_id` and returns the customer document, the list of purchased products (with quantities and spending), and the total amount spent per customer.

1D) This query creates a materialised “view” collection called `customer_sales_summary` that summarises sales per customer. It starts in the same way as 1C, using `$lookup` and `$unwind` to join orders with customers and products. In the first `$group`, I aggregate per `(customerId, productId)` to compute total quantity and revenue for each product, as well as the earliest and latest order dates and the set of order IDs. Then I group again by `customerId` to build one summary document per customer, summing the quantities and revenues across all products, collecting the distinct product categories, combining all order IDs, and tracking the first and most recent order dates. In the `$project` stage, I flatten the nested `order_ids` list using `$reduce` and `concatArrays`, and use `$size` to compute the total number of orders. Finally, the `$out` stage writes the result to a separate `customer_sales_summary` collection, so this aggregated view is stored in the database and can be queried directly without recomputing the whole pipeline.

2A.1) For this dataset, I modelled four main node types. A User node stores demographic information such as age, gender, and zip code, and each user is linked to exactly one Occupation node. A Movie node stores the movie’s basic metadata such as its title, release year, and IMDb link. Because each movie can belong to several genres, I created a separate Genre node for every genre name in the dataset. This avoids repeating genre attributes on each movie and keeps the graph more flexible, following the modelling guidance discussed in the lecture slides.

The relationship structure captures how users interact with movies and how movies are categorised. A `RATED` relationship connects a User to a Movie, with the rating value and timestamp stored as relationship properties. This is useful because ratings are actions rather than attributes of the user or the movie. Users are connected to their occupations through a `HAS_OCCUPATION` relationship, since occupations are shared across users and should not be duplicated. Finally, each Movie is connected to its Genre(s) through a `HAS_GENRE` relationship. I chose to model genres as separate nodes, rather than Booleans on movies, to reflect the many-to-many relationship and to simplify later similarity queries (e.g., comparing movies by shared genres).

This design keeps the graph simple and follows the principles from the lectures: avoid embedding many-to-many structures, store actions as relationships, and separate entities that are reused across the dataset. It also supports all the later tasks, such as recommending movies, identifying movie triangles, and computing similarity scores, because the structure makes traversal queries and pattern matching straightforward.



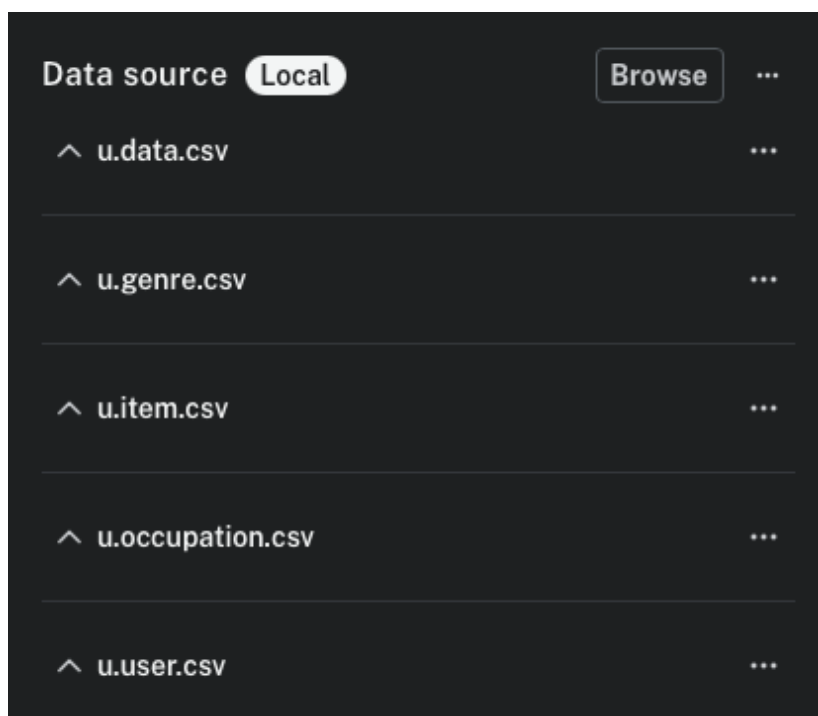
2A.2) To populate the graph, I first set up a Neo4j AuraDB Free instance and uploaded the MovieLens CSV files (u.user, u.data, u.item, u.genre, and u.occupation) into the Import section of Aura. Before loading the data, I added appropriate headers to the original files because the MovieLens version does not include them. The headers were taken from the dataset README and matched to each column manually. This made it possible for Neo4j's importer to recognise the fields as properties during loading.

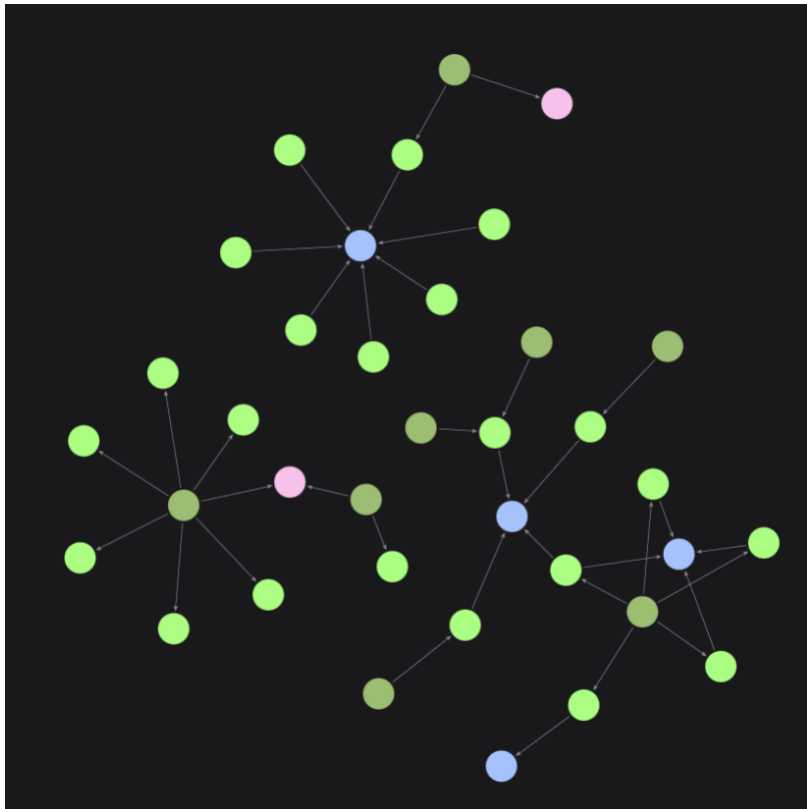
I mapped each CSV to a node type following the structure defined in 2A.1. The u.user.csv file was used to create User nodes; u.occupation.csv created Occupation nodes; u.item.csv created Movie nodes; and u.genre.csv created Genre nodes. The u.data.csv file was used to create the RATED relationships from User to Movie by matching userId and movieId. Because genre information is encoded as binary flags in u.item.csv, I did not use the importer to build HAS_GENRE. Instead, after the data load I created the relationships using a Cypher query that checks each movie's genre flags and links it to the corresponding Genre node. This step was necessary because the Importer cannot infer relationships from Boolean columns.

To verify the import worked correctly, I ran simple MATCH queries (e.g., MATCH (n) RETURN n LIMIT 25;) and checked the graph in the Explore view of AuraDB. This allowed me to confirm that the correct node labels and relationship types had been created. For the report, I included screenshots of:

1. the AuraDB instance after loading,
2. the graph model showing the four node types and the RATED / HAS_OCCUPATION / HAS_GENRE relationships, and
3. the Cypher commands used to generate the genre relationships.

Together, these demonstrate the full process of importing data into Neo4j and producing the final graph structure used in later tasks.





2B) For the recommendation query, I followed the same pattern as the restaurant recommendation example from the Neo4j lecture. I start from the target user, find other users who have rated the same movies (similar users), and then look at movies those similar users rated highly (rating ≥ 4). I exclude any movies the target user has already rated using a pattern predicate in the WHERE clause (NOT (u)-[:RATED]->(rec)). Finally, I aggregate over the remaining candidate movies, computing the average rating and the number of similar users who rated each movie, and return the top 10 by descending average rating.

2C) For 2C, I looked for “movie triangles,” which the assignment defines as triples of distinct movies sharing at least one genre and are all rated highly by the same users. To keep the computation feasible in AuraDB, I restricted the dataset to popular movies only (those with at least 40 ratings) and then limited the list to the top ~30 most rated films. This step is necessary because generating all possible triples grows very quickly and would otherwise exceed the memory limits.

After selecting this subset, I generated distinct triples using the condition `movie1.movieId < movie2.movieId < movie3.movieId`, which ensures that each triangle contains three different movies, avoiding permutations of the same set. After, I matched all Genre nodes connected to all three movies to ensure they genuinely share at least one genre. Finally, I filtered to users who rated every movie in the triple with rating ≥ 4 , which I used as my threshold for “highly rated.” The query returns the shared genre, the three movie titles, and the number of users who strongly liked all three, allowing us to identify connected groups of movies with a similar audience and amount of people that liked .

2D) For 2D, I computed movie similarity based purely on shared genres, following the corrected assignment instructions. Since most movies share at least one genre, I computed a `genreScore` equal to the number of genres two movies have in common. To avoid generating similarity edges for weak signals (e.g., movies sharing only one genre), I used a threshold of `genreScore  $\geq 3$` to represent “strong similarity.” This ensures the graph does not become overly dense and reflects more meaningful relationships.

As before, I limited the computation to the top ~30 most frequently rated movies to stay within AuraDB Free's memory limits and to focus on films with enough user engagement to produce reliable genre-based comparisons. For each pair of movies meeting the threshold, I created a `SIMILAR_TO` relationship storing the computed `genreScore`. The final query returns several examples of movie pairs with strong genre overlap and writes edges of the form `(:Movie)-[:SIMILAR_TO {genreScore: ...}]->(:Movie)` into the graph, as required by the task.